

Filtri Webcam in Real Time

Python + OpenCV

01 Descrizione del progetto

Cosa costruirete

Realizzerete un'applicazione desktop in Python che accede alla webcam in tempo reale e permette di applicare filtri, effetti visivi e overlay interattivi. L'utente controllerà tutto tramite tasti della tastiera mentre la webcam è accesa.

L'idea di base è semplice — avete già gli strumenti. La sfida è quante feature riuscite a far funzionare bene insieme in un'applicazione coerente e stabile.

 **Obiettivo principale:** non un singolo effetto bello, ma un'app completa con molte feature che funzionano in modo affidabile e si usano in modo intuitivo.

02 Requisiti minimi

Necessari per la sufficienza

Questi requisiti devono essere presenti e funzionanti. Senza di essi non si raggiunge la sufficienza, indipendentemente da quante feature aggiuntive ci sono.

1. Loop webcam stabile — finestra fluida, nessun crash, uscita pulita con il tasto Q e rilascio corretto della risorsa.
2. Almeno 3 filtri colore attivabili da tastiera — scala di grigi, negativo, e uno a scelta libera tra: sepia, solarizzazione, effetto termico (heatmap).
3. Sfondo sfocato — rilevare la faccia con Haar cascade e applicare GaussianBlur forte su tutto il frame tranne la regione della faccia, che resta nitida.
4. HUD sovrimpresso — visualizzare in ogni momento: nome del filtro attivo, numero di facce rilevate, FPS correnti calcolati in real time.
5. Screenshot con tasto S — salvare il frame corrente (con filtri applicati) come immagine .jpg con nome che include data e ora, in modo che i file non si sovrascrivano.

03 Feature aggiuntive

Determinano il voto finale

Non è richiesto implementarle tutte — è richiesto implementare bene quelle che scegliete. Una feature a metà o instabile vale meno di niente: penalizza invece di contribuire.

 **Attenzione** Una feature che crasha o si comporta in modo strano durante la demo viene considerata non implementata.

Filtri visivi

- Effetto fumetto (**cartoon**): bilateral filter ripetuto per appiattire i colori + bordi Canny sovrapposti in nero.
- Effetto termico (**heatmap**): scala di grigi + `cv2.applyColorMap` con `COLORMAP_JET` o `COLORMAP_INFERNO`.
- Pixelate: rimpicciolire il frame e riingrandirlo con interpolazione nearest — effetto pixel art.
- Vignettatura: scurire progressivamente i bordi con una maschera circolare sfumata costruita con NumPy.
- Flip specchio: capovolgimento orizzontale in real time, utile come 'selfie mode'.

Effetti sulla testa (richiedono face detection)

- Cappello: sovrapporre un PNG con canale alpha posizionato sopra il rettangolo della faccia, scalato in base alla sua dimensione.
- Occhiali: stesso principio del cappello ma posizionati all'altezza degli occhi usando la eye cascade.
- Maschera o barba: PNG sovrapposto nella metà inferiore del rettangolo della faccia.
- Etichetta personalizzata: scrivere un nome o un'etichetta sopra ogni faccia rilevata, configurabile nel codice.

Effetti di movimento

- Rilevamento movimento: confrontare frame consecutivi con `cv2.absdiff` e evidenziare con un colore le zone che cambiano oltre una certa soglia.
- Ghost effect: sovrapporre il frame corrente con una versione pesata del frame precedente — effetto scia.
- Motion blur simulato: applicare un blur direzionale con un kernel personalizzato che simula il mosso.

Interazione avanzata

- Barra filtri on-screen: mostrare nel frame in basso i filtri disponibili, evidenziando quello attivo — navigabile da tastiera.
- Registrazione video: avviare/fermare la registrazione del feed (con filtri applicati) su file .mp4 usando `cv2.VideoWriter`. Il tasto R avvia/ferma, un indicatore rosso segnala che si sta registrando.
- Modalità automatica: scorrere tutti i filtri attivi automaticamente ogni N secondi — configurabile.

04 Struttura del codice

Obbligatoria

Il codice deve essere organizzato in file separati per responsabilità. Non è accettato tutto in un unico blocco in `main.py`.

```
progetto/
├── main.py           # Loop principale, gestione tasti, orchestrazione
├── filters.py        # Funzioni per i filtri colore (grigio, negativo,
    cartoon...)
├── effects.py        # Effetti che usano face detection (sfondo blur,
    overlay...)
├── ui.py             # HUD, testo sullo schermo, barra filtri
├── assets/
│   ├── cappello.png # PNG con canale alpha per gli overlay
│   ├── occhiali.png
│   └── ...
```

Ogni funzione in `filters.py` e in `effects.py` deve rispettare queste regole:

- Riceve il frame come primo parametro
- Restituisce il frame modificato
- Non modifica l'originale — lavora su una copia se necessario
- Ha un commento in cima che spiega brevemente cosa fa

05 Deploy su sistemi esterni

Raspberry Pi e oltre

L'applicazione deve funzionare non solo sulla vostra macchina di sviluppo, ma anche su un sistema diverso — tipicamente un Raspberry Pi con webcam USB. Questa parte riguarda tutto quello che serve per far girare il progetto su una macchina che non avete mai configurato voi, senza spiegazioni a voce.

Cosa si intende per deploy

Deploy significa che un'altra persona, su un'altra macchina, deve poter clonare il vostro repository ed eseguire l'app in meno di 5 minuti, seguendo solo le istruzioni nel README. Nessun file mancante, nessuna dipendenza non documentata, nessun path hardcoded che funziona solo sul vostro PC.

requirements.txt — obbligatorio

Ogni dipendenza Python deve essere elencata con la versione esatta. Si genera in un comando:

```
pip freeze > requirements.txt
```

Sul Raspberry Pi (o qualsiasi altra macchina), l'installazione diventa:

```
pip install -r requirements.txt
```

Script di avvio — obbligatorio

Nessuno deve dover capire quale file eseguire o con quali parametri. Fornite uno script shell che fa tutto:

README — obbligatorio

Il README deve contenere almeno queste sezioni, scritte in modo che le capisca anche chi non ha mai visto il progetto:

- Descrizione in 3 righe — cosa fa l'app
- Requisiti — sistema operativo, versione Python, hardware necessario (webcam, ecc.)
- Installazione step by step — da zero, senza dare nulla per scontato

- Come avviare — comando esatto da eseguire
- Tasti disponibili — tabella chiara con tasto → azione
- Note specifiche per Raspberry Pi — se ci sono differenze rispetto al PC

GitHub Actions — per chi vuole il massimo

Se il progetto usa un server (es. il Raspberry Pi accede a un backend remoto, o si vuole un sistema di aggiornamento automatico), si può configurare una GitHub Action che esegue il deploy automaticamente ad ogni push sul branch main.

```
# .github/workflows/deploy.yml
name: Deploy su Raspberry Pi

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Deploy via SSH
        uses: appleboy/ssh-action@v1.0.0
        with:
          host: ${ secrets.PI_HOST }
          username: ${ secrets.PI_USER }
          key: ${ secrets.PI_SSH_KEY }
          script: |
            cd ~/progetto-opencv
            git pull origin main
            source venv/bin/activate
            pip install -r requirements.txt
```

 **Secrets:** le credenziali SSH (IP, utente, chiave privata) non vanno mai nel codice. Si configurano come GitHub Secrets nel repository — Settings → Secrets and variables → Actions. Il workflow le legge come variabili d'ambiente senza mai esporle.

Checklist deploy (da verificare prima della consegna)

- requirements.txt aggiornato e completo
- Script run.sh funzionante su una macchina pulita (testate su un PC diverso dal vostro)
- README con istruzioni complete e accurate
- Nessun path assoluto nel codice (es. /home/mario/progetto/assets/cappello.png)
- L'app si avvia correttamente su Raspberry Pi o su una VM Linux fresh
- Se usate GitHub Actions: i secrets sono configurati e la Action funziona

06 Valutazione

--

Criterio	Peso	Note
Requisiti minimi completi e funzionanti	40%	Base per la sufficienza
Feature aggiuntive (numero e qualità)	35%	Qualità > quantità
Qualità del codice (struttura, commenti)	15%	Leggibilità e organizzazione
Deploy corretto	10%	Live in classe

 **Nota:** un'app con 3 feature aggiuntive solide e codice pulito vale più di una con 8 feature instabili. La stabilità e la qualità del codice contano quanto il numero di effetti.

07 Consegna e presentazione

Consegna

- Repository GitHub pubblico con tutto il codice, gli asset, requirements.txt, run.sh e README
- Il repository deve essere clonabile e avviabile senza istruzioni aggiuntive a voce